SEV 게스트 런치 프로세스

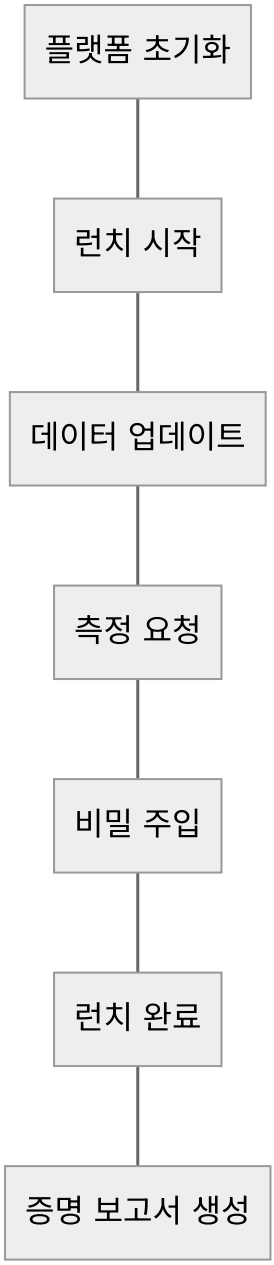
개요

AMD SEV (Secure Encrypted Virtualization) 게스트 런치 프로세스는 보안 가상 머신을 생성하고 실행하는 핵심 과정입니다. 이 문서는 SEV와 SEV-SNP 게스트 VM의 생성 과정을 단계별로 설명하고, 보안 정책 설정 및 메모리 암호화 설정 과정을 다룹니다.

SEV 게스트 런치 프로세스

1. 런치 프로세스 개요

SEV 게스트 런치는 다음과 같은 주요 단계로 구성됩니다:



2. 단계별 프로세스

2.1 플랫폼 초기화 (Platform Initialization)

게스트 런치를 시작하기 전에 SEV 플랫폼을 초기화해야 합니다.

SEV 초기화:

```
use sev::launch::sev::Launcher;


// SEV 런처 생성
```

```rust
let launcher = Launcher::new(kvm_fd, sev_fd)?;
```

SEV-ES 초기화:

```rust
// SEV-ES 런처 생성 (암호화된 상태 지원)
let launcher = Launcher::new_es(kvm_fd, sev_fd)?;
```

2.2 런치 시작 (Launch Start)

게스트 런치 프로세스를 시작하고 보안 정책을 설정합니다.

```rust
use sev::launch::sev::{Start, Policy, PolicyFlags, Session};
use sev::certs::sev::sev::Certificate;

// 보안 정책 설정
let policy = Policy {
flags: PolicyFlags::NO_DEBUG | PolicyFlags::ENCRYPTED_STATE,
minfw: Version::default(),
};

// 세션 정보 설정
let session = Session {
nonce: [0u8; 16],
wrap_tk: [0u8; 32],
wrap_iv: [0u8; 16],
```

```
wrap_mac: [0u8; 32],
policy_mac: [0u8; 32],
};

// 런치 시작 매개변수 구성
let start = Start {
policy,
cert: dh_certificate,
session,
};

// 런치 시작
let launcher = launcher.start(start)?;
```

2.3 데이터 업데이트 (Data Update)

게스트 메모리 영역을 암호화하고 등록합니다.

```
// 게스트 데이터 암호화 및 업데이트
let guest_data = &[/* 게스트 이미지 데이터 */];
launcher.update_data(guest_data)?;

// SEV-ES의 경우 VMSA 업데이트
```

```
launcher.update_vmsa()?;
```

2.4 측정 요청 (Measurement Request)

게스트의 무결성 측정값을 요청합니다.

```
// 측정 요청
let launcher = launcher.measure()?;


// 측정값 확인
let measurement = launcher.measurement();
println!("게스트 측정값: {:?}", measurement.measure);
```

2.5 비밀 주입 (Secret Injection)

증명 과정 후 게스트에 비밀 정보를 주입합니다.

```
use sev::launch::sev::{Secret, Header, HeaderFlags};


// 비밀 헤더 구성
let header = Header {
flags: HeaderFlags::default(),
iv: [0u8; 16],
mac: [0u8; 32],
};
```

```
// 비밀 데이터 구성
let secret = Secret {
header,
ciphertext: encrypted_secret_data,
};

// 비밀 주입
let guest_address = 0x1000; // 게스트 메모리 주소
launcher.inject(&secret, guest_address)?;
```

2.6 런치 완료 (Launch Finish)

게스트 런치 프로세스를 완료합니다.

```
// 런치 완료 (핸들 반환)
let handle = launcher.finish()?;

// 또는 증명 가능한 런처로 완료
let launcher = launcher.finish_attestable()?;
```

3. 보안 정책 설정

3.1 정책 플래그

SEV 게스트의 보안 정책은 다음 플래그로 제어됩니다:

```rust
use sev::launch::sev::PolicyFlags;


let  policy_flags = PolicyFlags::NO_DEBUG // 디버깅 금지
| PolicyFlags::NO_KEY_SHARING // 키 공유 금지
| PolicyFlags::ENCRYPTED_STATE // SEV-ES 보호 필요
| PolicyFlags::NO_SEND // 다른 플랫폼 전송 금지
| PolicyFlags::DOMAIN // 도메인 내 전송만 허용
| PolicyFlags::SEV; // SEV 지원 플랫폼만 허용
```

3.2 정책 설명

- **NO_DEBUG**: 게스트 디버깅을 금지합니다
- **NO_KEY_SHARING**: 다른 게스트와 키 공유를 금지합니다
- **ENCRYPTED_STATE**: SEV-ES 보호를 요구합니다
- **NO_SEND**: 게스트를 다른 플랫폼으로 전송하는 것을 금지합니다
- **DOMAIN**: 도메인 외부로의 전송을 금지합니다
- **SEV**: SEV를 지원하지 않는 플랫폼으로의 전송을 금지합니다

4. 메모리 암호화 설정

4.1 암호화 영역 등록

```rust
// KVM 암호화 영역 등록
launcher.register_kvm_enc_region(memory_region)?;
```

```
// 등록 없이 데이터 업데이트 (특수한 경우)

launcher.update_data_without_registration(data)?;
```

4.2 메모리 보호 메커니즘

SEV는 다음과 같은 메모리 보호 메커니즘을 제공합니다:

1. **메모리 암호화**: 게스트 메모리가 하드웨어 수준에서 암호화됩니다
2. **무결성 보호**: 메모리 내용의 무결성이 보장됩니다
3. **접근 제어**: 하이퍼바이저의 게스트 메모리 접근이 제한됩니다

SEV-SNP 게스트 런치 프로세스

1. SNP 런치 프로세스 개요

SEV-SNP는 기존 SEV보다 향상된 보안 기능을 제공합니다:

2. SNP 단계별 프로세스

2.1 SNP 초기화

```
use sev::launch::snp::Launcher;

// SEV-SNP 런처 생성
let  launcher = Launcher::new(vm_fd, sev_fd)?;
```

2.2 SNP 런치 시작

```
use sev::launch::snp::Start;
```

```rust
use sev::firmware::guest::GuestPolicy;


// 게스트 정책 설정
let policy = GuestPolicy::default();


// GOSVW (Guest OS Visible Workarounds) 설정
let gosvw = [0u8; 16];


// SNP 런치 시작
let start = Start::new(policy, gosvw);
let launcher = launcher.start(start)?;
```

2.3 페이지 업데이트

```rust
use sev::launch::snp::{Update, PageType};


// 일반 데이터 페이지 업데이트
let update = Update::new(
start_gfn, // 게스트 프레임 번호
guest_data, // 게스트 데이터
PageType::Normal // 페이지 타입
);
```

```
let gpa = 0x1000; // 게스트 물리 주소
let gpa_len = 0x1000; // 길이

launcher.update_data(update, gpa, gpa_len)?;
```

2.4 페이지 타입

SNP는 다양한 페이지 타입을 지원합니다:

```
use sev::launch::snp::PageType;

// 페이지 타입별 용도
let page_types = [
PageType::Normal, // 일반 데이터 페이지
PageType::Vmsa, // VMSA 페이지
PageType::Zero, // 제로 페이지
PageType::Unmeasured, // 측정되지 않은 페이지
PageType::Secrets, // 비밀 저장 페이지
PageType::Cpuid, // CPUID 정보 페이지
];
```

2.5 SNP 런치 완료

```
use sev::launch::snp::Finish;
```

```rust
// 호스트 데이터 설정
let host_data = [0u8; 32];


// ID 블록 및 인증 정보 (선택사항)
let id_block = Some(&id_block_data[..]);
let id_auth = Some(&id_auth_data[..]);


// 런치 완료
let finish = Finish::new(id_block, id_auth, host_data);
let (vm_fd, sev_fd) = launcher.finish(finish)?;
```

SEV vs SEV-SNP 차이점

1. 주요 차이점

기능	SEV	SEV-SNP
메모리 보호	암호화만	암호화 + 무결성
페이지 단위 보호	제한적	완전 지원
원격 증명	기본적	고급 증명
VMPL 지원	없음	4단계 권한
RMP 지원	없음	하드웨어 RMP

2. 보안 향상사항

2.1 메모리 무결성 보호

SNP는 메모리 암호화뿐만 아니라 무결성 보호도 제공합니다:

```
// SNP에서는 메모리 속성을 명시적으로 설정
KvmSetMemoryAttributes::new(
gpa,
gpa_len,
KVM_MEMORY_ATTRIBUTE_PRIVATE
).set_attributes(&mut vm_fd)?;
```

2.2 RMP (Reverse Map Table) 보호

SNP는 하드웨어 RMP를 통해 페이지 단위 보호를 제공합니다.

2.3 VMPL (VM Permission Levels)

```
use sev::launch::snp::VmplPerms;

// VMPL 권한 설정
let vmpl_perms = VmplPerms::READ
| VmplPerms::WRITE
| VmplPerms::EXECUTE_USER
| VmplPerms::EXECUTE_SUPERVISOR;
```

오류 처리 및 디버깅

1. 일반적인 오류 상황

1.1 플랫폼 초기화 실패

```
match Launcher::new(kvm_fd, sev_fd) {
Ok(launcher) => launcher,
Err(FirmwareError::KnownSevError(SevError::InvalidPlatformState)) => {
// 플랫폼 상태 확인 및 재초기화 필요
eprintln!("플랫폼 상태가 올바르지 않습니다");
return Err("플랫폼 재초기화 필요".into());
},
Err(e) => return Err(e.into()),
}
```

1.2 정책 설정 오류

```
// 정책 검증
if  policy.flags.contains(PolicyFlags::ENCRYPTED_STATE) {
// SEV-ES 기능이 필요한 경우 확인
if !platform_supports_es() {
return Err("플랫폼이 SEV-ES를 지원하지 않습니다".into());
}
}
```

1.3 메모리 암호화 오류

```
match launcher.update_data(guest_data) {
Ok(_) => {},
Err(FirmwareError::KnownSevError(SevError::InvalidAddress)) => {
eprintln!("잘못된 메모리 주소입니다");
return Err("메모리 주소 확인 필요".into());
},
Err(e) => return Err(e.into()),
}
```

2. 디버깅 팁

2.1 측정값 검증

```
// 예상 측정값과 비교
let expected_measurement = calculate_expected_measurement(&guest_image);
let actual_measurement = launcher.measurement();

if expected_measurement != actual_measurement.measure {
eprintln!("측정값이 일치하지 않습니다");
eprintln!("예상: {:?}", expected_measurement);
eprintln!("실제: {:?}", actual_measurement.measure);
}
```

2.2 로그 활성화

```rust
// 디버그 빌드에서 상세 로그 출력
#[cfg(debug_assertions)]
{
env_logger::init();
log::debug!("SEV 런치 프로세스 시작");
}
```

모범 사례

1. 보안 고려사항

1. **정책 설정**: 가능한 한 제한적인 정책을 사용하세요
2. **측정 검증**: 항상 게스트 측정값을 검증하세요
3. **비밀 관리**: 비밀 정보는 안전하게 암호화하여 주입하세요
4. **오류 처리**: 모든 오류 상황을 적절히 처리하세요

2. 성능 최적화

1. **메모리 정렬**: 페이지 경계에 맞춰 메모리를 정렬하세요
2. **배치 처리**: 가능한 한 데이터를 배치로 처리하세요
3. **리소스 관리**: 파일 디스크립터와 메모리를 적절히 관리하세요

3. 호환성 고려사항

1. **플랫폼 확인**: 대상 플랫폼의 SEV 지원 여부를 확인하세요
2. **펌웨어 버전**: 최소 펌웨어 버전 요구사항을 확인하세요

3. **기능 지원**: 사용하려는 기능이 지원되는지 확인하세요

결론

SEV 게스트 런치 프로세스는 보안 가상화의 핵심 과정입니다. 올바른 단계를 따르고 적절한 보안 정책을 설정함으로써 안전하고 신뢰할 수 있는 가상 머신을 생성할 수 있습니다. SEV-SNP는 추가적인 보안 기능을 제공하므로, 높은 보안이 요구되는 환경에서는 SNP 사용을 권장합니다.